Vulcan

Supplementary slides. AMD AI DevMaster Hackathon, Track 2, Private AI Agents.

A codebase agent that generates every token on hardware you control. One section per slide. Every figure here is reproducible from the committed JSON in `bench-results/` with `vulcan bench-compare`, so almost nothing in these slides was typed in by hand.

1. The problem

Developers on private or regulated codebases cannot paste source into a cloud LLM. So they either stop using AI assistance, or they leak.

Vulcan is a codebase agent that generates every token on hardware you control, never a third-party LLM API.

2. What it does

Index a repo, then ask it questions in plain language. It searches, reads files, greps, runs tests and answers with `file:line` citations.

Autonomous tool use over a ReAct loop, persistent per-project memory, zero data egress.

3. Architecture

```
user -- CLI -- Agent (ReAct, JSON tool protocol)
                |- RAG index (SQLite + cosine, no vector DB)
                |- Tools (search_code, read_file, grep, run_cmd*, write_file)
                |- Memory (durable per-project notes)
                `- LLM client -- OpenAI-compatible endpoint
                                   `- vLLM on ROCm / Radeon GPU
```

`*` allowlisted commands only, file access is path-jailed.

One environment variable switches the backend. The same agent runs on Ollama during development and vLLM-ROCm in production, which is what made the measurements in slide 5 a one-variable swap at the client. The two backends do not serve identical weights, which spec.md section 4.2 sets out in full.

4. Running on Radeon

vLLM 0.16.1 on ROCm 7.2.1, serving `Qwen/Qwen3-8B` at `--gpu-memory-utilization 0.92`.

The agent needs no code change to target it: `vulcan/llm.py` speaks OpenAI-compatible chat against any `base_url`, with a separate client for embeddings so the two can point at different servers.

One honest caveat: `vllm serve Qwen/Qwen3-8B` runs task=generate and exposes no `/v1/embeddings` route (verified, 404). Radeon Cloud allows one active instance per account, so the measured setup runs generation on the GPU and embeddings on a separate local model. Both are operator-controlled, so no source leaves your machines, but only generation is GPU-served here.

5. Concurrency is the whole argument

A single-stream benchmark hides the thing that matters. An agent fleet issues many ReAct steps at once.

concurrent	laptop (Ollama 4B)	Radeon (vLLM 8B)
1	28.4 chunks/s, TTFT 4.36s	23.5 chunks/s, TTFT 1.49s
2	**20.0** chunks/s, TTFT **42.21s**	**42.9** chunks/s, TTFT **1.25s**
4	not run, curve already inverted	87.0 chunks/s, TTFT 0.55s
8	not run	**179.2** chunks/s, TTFT **0.41s**

- Radeon: **7.63x throughput across an 8x load increase, 95% efficiency.**

The control: what actually produces the scaling

Stock settings invite a fair question, so the claim was tested by taking batching away. `--max-num-seqs 4` caps the batch at four sequences; everything else identical.

concurrent	stock	capped at 4
1	20.0 chunks/s	21.6
4	78.1	86.7
8	**162.5**	**88.0**

8.12x stock, 4.07x capped at 4 — the capped run scales to its own cap.

That is the control the claim needed: the scaling is continuous batching, not clock or bandwidth. Past the cap, throughput falls 46% and median TTFT goes from 1.6s to **25.5s**. So the stock config is already right for this workload, and this is the measurement that says so rather than an assumption.

TTFT *improves* 3.6x. Batch wall clock moves only 51s to 65s for 1 request or 8, against 8x the work.

- Laptop: one extra request makes aggregate throughput *fall* and TTFT grow nearly 10x. No continuous batching, so requests queue instead of sharing.

At two concurrent requests: **1.2s against 42.2s**, a 34x gap in what the user waits, with the Radeon carrying twice the parameters.

Measured on two independent clients, both with committed JSON: **6.96x** (`demo-live.json` , recorded on camera) and **7.63x** (`radeon-vllm-concurrency.json`). An earlier 1-to-8 run measured 7.23x but its

JSON was not persisted, so it is not counted here. The 6.96x run is the one visible on screen in the demo video, produced by the shipped CLI rather than by a benchmarking harness written for the occasion.

Quantization: measured, and rejected

AWQ 4-bit on the same instance, same flags, same harness. It loads and serves on **gfx1100 under ROCm vLLM**, which is not a given.

concurrent	fp16	AWQ 4-bit
1	20.0 chunks/s	14.4
8	**162.5**	**111.1**

0.68x. Decode agrees at 0.65x.

AWQ buys memory by paying compute, and on a 48 GB card serving an 8B model there is no memory problem to buy out of. So the dequantization cost is paid for a benefit that is not needed. It is the right call on a smaller card, or past about 30B where fp16 stops fitting. This measurement is what tells you which regime you are in.

5b. Two more measured wins

Thinking mode off. Qwen3 reasons by default, which is the wrong trade for an agent: every step pays for a preamble nobody reads.

mode	deltas	wall clock	rate
thinking (default)	4058	170.9s	23.8/s
`enable_thinking: false`	1168	49.0s	24.0/s

Identical rate, 3.5x fewer tokens, so 3.5x less latency per step. A serving-config win, not a faster GPU. Reproducible with `VULCAN_ENABLE_THINKING=false`.

Prefill has a cliff. 512 words costs 0.316s, 2048 costs 0.399s, 8192 costs 2.091s. So the RAG layer has a budget: stay under ~2000 words of retrieved context and TTFT stays under half a second.

6. Honest measurement

- Radeon numbers are measured over an HTTPS proxy (the instance SSH port is not reachable from the client network), so the proxy round-trip is inside every reported TTFT. It was measured separately and is stated.
- "Tokens/sec" counts streamed SSE deltas, not tokenizer tokens.
- Cold start is reported, not hidden: the first two repetitions of each prompt carry cache-warming cost. Repeat counts are per run and recorded in each JSON file: 5 for the Radeon decode set, 3 for prefill, 2 for the laptop decode baseline, 1 per level for concurrency.

7. What is next

- Quantization sweep: fp16 against int8/awq, quality against speed
- vLLM tuning: max-num-seqs, chunked prefill
- Embedding throughput on GPU against CPU